

Fundamentos de redes neuronales artificiales

Apuntes sobre keras

Juan Miguel Santos
Emiliano Churruca

LIDEC
Universidad Nacional de Hurlingham

(6 de mayo de 2025)

Redes neuronales artificiales

Introducción

En este apunte trataremos de contestar algunas preguntas sobre keras

- ¿Qué es keras?
- ¿Qué es TensorFlow?
- ¿Cuáles son los modelos de rna que se pueden hacer con keras?
- ¿Cómo implementar un perceptron simple con keras?
- ¿Cómo implementar un perceptron multicapa?

Redes neuronales artificiales

Introducción

En este apunte trataremos de contestar algunas preguntas sobre keras (cont)

- ¿Cómo inicializar el valor de los pesos en la red?
- ¿Cómo obtener los estados de activación de las neuronas en las capas ocultas?
- ¿Cómo graficar la evolución de la función de pérdida durante el entrenamiento?
- ¿Cómo graficar la evolución de las métricas durante el entrenamiento?

Redes neuronales artificiales

Introducción

En este apunte trataremos de contestar algunas preguntas sobre keras (cont)

- ¿Cómo grabar una red entrenada?
- ¿Cómo recuperarla?
- ¿Cómo obtener los pesos?

Redes neuronales artificiales

¿Qué es keras?

- Keras es una librería (con licencia MIT) escrita en Python.
- Está basada en el trabajo de François Chollet, un desarrollador de Google, quien trabajó en el proyecto ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System).
- La primera versión es del 28 de marzo de 2015.
- La idea fue disponer de una herramienta para la creación de redes neuronales artificiales.

Redes neuronales artificiales

¿Qué es keras?

- Trabaja sobre los backends Theano, Microsoft Cognitive Toolkit y TensorFlow.
- keras fue pensado para que su desarrollo siga creciendo independientemente del software de base.
- Trabaja a nivel de modelo y no resolviendo las cosas que hacen los back-ends, por ejemplo, operaciones entre vectores, matrices (o tensores, ya se verá qué son los tensores), etc.

Redes neuronales artificiales

¿Qué es TensorFlow?

- Tensorflow es una librería desarrollada por Google. Surge en el contexto del proyecto Google Brain.
- Es multiplataforma. Puede trabajar con GPUs, CPUs y TPUs (unidades de procesamiento de tensores).
- Aparte de la API para Python, TensorFlow tiene librerías en C++, C#, Java y Rust.

Redes neuronales artificiales

¿Qué es TensorFlow?

- Una TPU es un circuito integrado especializado (ASIC, accelerator application-specific integrated circuit).
- Según Wikipedia: "La primera generación de las TPU de Google se presentó en la conferencia I/O 2016, diseñado específicamente para ejecutar redes neuronales entrenadas. Estas TPU tienen menos precisión en comparación con los cálculos realizados en una CPU o GPU normales, pero es suficiente para los cálculos que tienen que realizar."

Redes neuronales artificiales

¿Qué es TensorFlow?

- El nombre viene de flujo de tensores (a diferencia del flujo de control clásico de un programa imperativo).
- Los tensores son estructuras de datos y la forma fácil de visualizarlo es pensando a partir del tipo de variables que conocemos.
- Por ejemplo, un vector tiene una dimensión, una matriz tiene dos dimensiones, un tensor tiene n dimensiones.

Redes neuronales artificiales

¿Qué es TensorFlow?

- Un ejemplo de tensor es una imagen en color. Supongamos que la imagen es de 1920×1080 . Como la imagen tiene 3 capas, rojo, verde y azul, la imagen se puede representar usando un tensor de 3 dimensiones: $1920 \times 1080 \times 3$.

Redes neuronales artificiales

¿Cuáles son los modelos de rna que se pueden hacer con keras?

- Hay dos tipos de modelos: el secuencial y el funcional.
- En el secuencial, las capas se van definiendo y se apilan una después de la otra.
- El modelo funcional, se basa en la idea de que un modelo de aprendizaje profundo suele ser un gráfico acíclico dirigido (DAG) de capas.
- La API funcional es un conjunto de herramientas para construir gráficos de capas.

Redes neuronales artificiales

¿Cuáles son los modelos de rna que se pueden hacer con keras?

- En el modo funcional, las capas se van agregando indicando desde donde van los datos (desde qué capa) hasta donde llegan (hasta qué capa).

Redes neuronales artificiales

¿Cuáles son los modelos de rna que se pueden hacer con keras?

- Ejemplo (extraído de <https://www.tensorflow.org/guide/keras/functional?hl=es-419>):

```
inputs = keras.Input(shape=(784,), name='img')
x = layers.Dense(64, activation='relu')(inputs)
x = layers.Dense(64, activation='relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)
model = keras.Model(inputs=inputs, outputs=outputs,
name='mnist_model')
```

Redes neuronales artificiales

¿Cuáles son los modelos de rna que se pueden hacer con keras?

- Para compilar, ajustar o evaluar un modelo hecho de forma funcional es igual a como hacemos con el modo secuencial.
- La forma de ver una rna como un gráfico acíclico dirigido donde los nodos son capas permite, por ejemplo, que dos capas diferentes sean entrada de otra capa, o al revés, que una misma capa sea entrada de dos capas diferentes.

Redes neuronales artificiales

¿Cómo implementar un perceptron simple con keras?

```
# crea el modelo con una salida y dos entradas  
activation='sigmoid'  
model = Sequential()  
model.add(Dense(1, input_dim = 2, activation='sigmoid'))  
# Compila el modelo  
opt = keras.optimizers.SGD(learning_rate = 0.1, momentum =  
0.0)  
model.compile(loss = 'mean_squared_error', optimizer = opt,  
metrics = ['accuracy'])  
# Ajusta el modelo  
model.fit(X, Y, epochs = 200, batch_size = 1)
```

Redes neuronales artificiales

¿Cómo implementar un perceptron multicapa con keras?

```
# crea el modelo con una salida y dos entradas  
# Crear el modelo  
model = Sequential()  
model.add(Dense(5, input_dim = 2, activation = 'tanh'))  
model.add(Dense(1, activation = 'sigmoid', name='salida'))  
# Compilar el modelo  
opt = keras.optimizers.Adam(learning_rate = 0.01)  
metrica = keras.metrics.BinaryAccuracy(name="binary_accuracy",  
dtype=None, threshold=0.5)  
model.compile(loss = 'MSE', optimizer = opt, metrics = [metrica])  
# Ajustar el modelo  
historia = model.fit(X, Y, epochs = 300, batch_size = 1)
```


¿Cómo inicializar el valor de los pesos en la red?

Se lleva a cabo cuando se define cada capa. Para eso es necesario importar “initializers” de keras:

- `from keras import layers`
- `from keras import initializers`

¿Cómo inicializar el valor de los pesos en la red?

Luego, cuando se define una capa se define la forma de inicializarlos.

Por ejemplo:

```
capa = layers.Dense( units = 64,  
kernel_initializer = initializers.RandomNormal(stddev = 0.01),  
bias_initializer = initializers.Zeros() )
```

¿Cómo inicializar el valor de los pesos en la red?

O

```
modelo.add(Dense(5, input_dim = 2,  
activation = 'tanh', name='oculta_1',  
kernel_initializer = initializers.RandomNormal(mean = 0.0,  
stddev = 0.01), bias_initializer = initializers.Zeros() ) )
```

¿Cómo inicializar el valor de los pesos en la red?

También se puede usar inicializadores estándar como “random_normal” o “zeros”.

Por ejemplo:

```
capa = layers.Dense( units=64,  
kernel_initializer='random_normal', bias_initializer='zeros')
```

Hay muchas alternativas para inicializar los pesos de una capa:

- Normal
inicializador = RandomNormal(mean = 0.0, stddev = 1.0)
capa = Dense(3, kernel_initializer = inicializador)
- Uniforme
inicializador = RandomUniform(minval = 0.0, maxval = 1.0)
capa = Dense(3, kernel_initializer = inicializador)
- Con ceros (también con Ones())
inicializador = Zeros()
capa = Dense(units = 3, kernel_initializer = inicializador)

Hay muchas alternativas para inicializar los pesos de una capa:

- Glorot normal

```
inicializador = GlorotNormal()
```

```
capa = Dense(3, kernel_initializer = inicializador)
```

Hay muchas alternativas para inicializar los pesos de una capa:

- Esta forma asigna valores al azar siguiendo una normal con media cero.
- Para la desviación estándar tiene en cuenta con cuántas neuronas de la capa anterior está conectada cada unidad de la capa (fan_in) y a cuántas unidades está conectada de la capa superior (fan_out):

$$\text{desviacion_estándar} = \sqrt{2 / (fan_in + fan_out)}$$

donde fan_in y fan_out están definidos por las dimensiones del tensor de los pesos de la capa y la dimensión del tensor de los pesos de la capa próxima.

Hay muchas alternativas para inicializar los pesos de una capa:

- HeNormal

Similar a Glorot pero solo tiene en cuenta el `fan_in` (lo cual parece algo más racional):

`inicializador = HeNormal()`

`capa = Dense(3, kernel_initializer = inicializador)`

con

$$stddev = \sqrt{\frac{2}{fan_in}}$$

- Valor constante

`inicializador = Constant(10.)`

`capa = Dense(3, kernel_initializer = inicializador)`

¿Cómo obtener los estados de activación de las neuronas en las capas ocultas?

Durante la creación del modelo, indicar un nombre para la capa para la cual se desea conocer el estado de activación de sus unidades.

```
# Crear el modelo
model = Sequential()
model.add(Dense(5, input_dim = 2,
                activation = 'tanh', name = 'oculta_1'))
model.add(Dense(1, activation = 'sigmoid',
                name = 'salida'))

.....
```

¿Cómo obtener los estados de activación de las neuronas en las capas ocultas?

Durante la creación del modelo, indicar un nombre para la capa para la cual se desea conocer el estado de activación de sus unidades.

```
nombre_de_la_capa = 'oculta_1'  
intermediate_layer_model = keras.Model(inputs =  
    model.input, outputs = model.get_layer  
    (nombre_de_la_capa).output)
```

¿Cómo obtener los estados de activación de las neuronas en las capas ocultas?

Estado de activación cuando presentamos el primer ejemplo

```
X_1 = numpy.array([X[1, :]])
```

```
activaciones_capa = intermediate_layer_model.predict(X_1)
```

```
print(activaciones_capa)
```

¿Cómo graficar la evolución de la función de pérdida durante el entrenamiento?

```
import matplotlib.pyplot as plt
.....
historia = model.fit(X, Y, epochs = 300, batch_size = 1)
# Graficar loss
plt.plot(historia.history['loss'])
plt.title('Función de pérdida durante el entrenamiento')
plt.ylabel('función de pérdida')
plt.xlabel('épocas')
plt.legend(['entrenamiento'], loc='upper right')
plt.show()
```

¿Cómo graficar la evolución de las métricas durante el entrenamiento?

```
import matplotlib.pyplot as plt
.....
historia = model.fit(X, Y, epochs = 300, batch_size = 1)
# Graficar exactitud (accuracy)
plt.plot(historia.history['accuracy'])
plt.title('Exactitud (accuracy) durante el entrenamiento')
plt.ylabel('Exactitud')
plt.xlabel('épocas')
plt.legend(['entrenamiento'], loc='upper right')
plt.show()
```

¿Cómo grabar una red entrenada?

Una vez entrenada, se puede guardar el modelo siguiendo:
`model.save('./pmc_xor.keras')`

¿Cómo recuperarla?

Si se ha grabado un modelo, se puede leer siguiendo:

```
model = keras.models.load_model('./pmc_xor.keras')
```

¿Cómo obtener los pesos?

Consideremos que ya hemos entrenado la red y la hemos guardado en el archivo `pmc_xor.keras`

Leemos el modelo:

```
model = keras.models.load_model('./pmc_xor.keras')
esta_capa = model.layers[0]
print(" ")
print("esta capa:")
print(esta_capa)
print(" ")
```


¿Cómo obtener los pesos?

```
estos_pesos = esta_capa.get_weights()[0]  
print(" ")  
print("Pesos:")  
print(estos_pesos)  
print(" ")
```

¿Cómo obtener los pesos?

```
estos_sesgos = esta_capa.get_weights()[1]  
print(" ")  
print("Sesgos:")  
print(estos_sesgos)  
print(" ")
```

¿Cómo obtener los pesos?

```
configuracion = esta_capa.get_config()  
print(" ")  
print("Configuracion:")  
for clave in configuracion:  
    print(clave, configuracion[clave])
```